

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I M.Devi Srilatha (192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Identify Functional Dependencies and Determine Candidate Keys

Aim

To identify all functional dependencies in the relation **STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)** and determine its candidate keys.

Theory

A Functional Dependency (FD) is a constraint between two sets of attributes in a relation. It describes a relationship where the value of one attribute or a group of attributes uniquely determines the value of another attribute.

A functional dependency is represented as:

$$X \rightarrow Y$$

Here, X is called the **determinant** and Y is called the **dependent attribute**. The notation $X \rightarrow Y$ means that if two tuples have the same value for X, they must also have the same value for Y.

Consider the relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Where:

- **SID** – Student ID
- **SName** – Student Name
- **CID** – Course ID
- **CName** – Course Name
- **Instructor** – Course Instructor
- **Grade** – Grade obtained by the student

Assume that every student has a unique SID and every course has a unique CID.

Therefore, the functional dependencies are:

$$\mathbf{SID \rightarrow SName}$$

A student ID uniquely determines the student name.

$$\mathbf{CID \rightarrow CName}$$

A course ID uniquely determines the course name.

$$\mathbf{CID \rightarrow Instructor}$$

A course ID uniquely determines its instructor.

$(SID, CID) \rightarrow Grade$

The grade depends on both the student and the course because the same student can take different courses and obtain different grades.

Therefore:

$(SID, CID) \rightarrow SName, CName, Instructor, Grade$

Thus, the combination of SID and CID determines every attribute in the relation.

Candidate Key

A **candidate key** is a minimal set of attributes that uniquely identifies every tuple in a relation.

Consider:

$(SID, CID)^+$

Starting with SID and CID:

$(SID, CID)^+$
= {SID, CID}
= {SID, CID, SName}
= {SID, CID, SName, CName, Instructor}
= {SID, CID, SName, CName, Instructor, Grade}

Therefore:

$(SID, CID)^+ = \text{All attributes}$

Neither SID nor CID alone can determine all attributes.

Hence:

Candidate Key = {SID, CID}

Result

The functional dependencies are:

$SID \rightarrow SName$
 $CID \rightarrow CName$
 $CID \rightarrow Instructor$
 $(SID, CID) \rightarrow Grade$

The candidate key is:

(SID, CID) .

```
mysql> DESCRIBE STUDENT_COURSE;
```

Field	Type	Null	Key	Default	Extra
SID	int	NO	PRI	NULL	
SName	varchar(50)	YES		NULL	
CID	int	NO	PRI	NULL	
CName	varchar(50)	YES		NULL	
Instructor	varchar(50)	YES		NULL	
Grade	varchar(5)	YES		NULL	

6 rows in set (0.03 sec)

```
mysql> SELECT * FROM STUDENT_COURSE;
```

SID	SName	CID	CName	Instructor	Grade
101	Ravi	201	DBMS	Dr. Rao	A
101	Ravi	202	Python	Dr. Sharma	B
102	Priya	201	DBMS	Dr. Rao	A
102	Priya	203	AI	Dr. Kumar	A
103	Kiran	202	Python	Dr. Sharma	B

5 rows in set (0.00 sec)

```
mysql> DESCRIBE STUDENT_COURSE;
```

Field	Type	Null	Key	Default	Extra
SID	int	NO	PRI	NULL	
SName	varchar(50)	YES		NULL	
CID	int	NO	PRI	NULL	
CName	varchar(50)	YES		NULL	
Instructor	varchar(50)	YES		NULL	
Grade	varchar(5)	YES		NULL	

6 rows in set (0.01 sec)

```
mysql> SELECT SID, CID, COUNT(*)  
-> FROM STUDENT_COURSE  
-> GROUP BY SID, CID  
-> HAVING COUNT(*) > 1;  
Empty set (0.00 sec)
```

```
mysql> SELECT SID, CID, COUNT(*)  
-> FROM STUDENT_COURSE  
-> GROUP BY SID, CID  
-> HAVING COUNT(*) > 1;  
Empty set (0.00 sec)
```

```
mysql> DESCRIBE STUDENT_COURSE;
```

Field	Type	Null	Key	Default	Extra
SID	int	NO	PRI	NULL	
SName	varchar(50)	YES		NULL	
CID	int	NO	PRI	NULL	
CName	varchar(50)	YES		NULL	
Instructor	varchar(50)	YES		NULL	
Grade	varchar(5)	YES		NULL	

6 rows in set (0.01 sec)

2. Convert an Unnormalized Relation into 1NF

Aim

To demonstrate the conversion of an unnormalized relation into **First Normal Form (1NF)** with proper justification.

Theory

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The first stage of normalization is **First Normal Form (1NF)**.

A relation is in **1NF** if:

1. Every attribute contains only atomic values.
2. There are no repeating groups.
3. Each cell contains a single value.
4. Each row is uniquely identifiable.

Unnormalized Relation

Consider:

SID	SName	Courses
101	Ravi	DBMS, Python
102	Anu	AI, Java

The `Courses` attribute contains multiple values in a single cell.

For example:

`Courses = DBMS, Python`

Therefore, the relation is **not in 1NF**.

Conversion to 1NF

The repeating values must be separated into individual rows.

The relation becomes:

SID	SName	Course
101	Ravi	DBMS
101	Ravi	Python
102	Anu	AI
102	Anu	Java

Now each cell contains only one value.

For example:

```
101 | Ravi | DBMS
101 | Ravi | Python
```

Each value is atomic.

Step-by-Step Justification

Step 1: Identify the repeating group.

The `Courses` attribute contains multiple values.

Step 2: Remove the repeating group.

Separate each course into a separate row.

Step 3: Ensure atomicity.

Every attribute now contains only one value.

Step 4: Identify a suitable key.

The combination of **SID and Course** can uniquely identify each enrollment.

Therefore:

Candidate Key = (SID, Course)

Result

The unnormalized relation was successfully converted into **1NF** by removing repeating groups and ensuring that all attributes contain atomic values.

```
mysql> CREATE TABLE STUDENT_COURSE_1NF (
->     SID INT,
->     SName VARCHAR(50),
->     Course VARCHAR(50),
->     Instructor VARCHAR(50),
->     PRIMARY KEY (SID, Course)
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO STUDENT_COURSE_1NF VALUES
-> (101, 'Ravi', 'DBMS', 'Dr. Rao'),
-> (101, 'Ravi', 'Python', 'Dr. Sharma'),
-> (102, 'Priya', 'AI', 'Dr. Kumar'),
-> (102, 'Priya', 'DBMS', 'Dr. Rao'),
-> (103, 'Kiran', 'Python', 'Dr. Sharma');
```

Query OK, 5 rows affected (0.01 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
mysql> DESCRIBE STUDENT_COURSE_1NF;
```

Field	Type	Null	Key	Default	Extra
SID	int	NO	PRI	NULL	
SName	varchar(50)	YES		NULL	
Course	varchar(50)	NO	PRI	NULL	
Instructor	varchar(50)	YES		NULL	

4 rows in set (0.01 sec)

```
mysql> DESCRIBE STUDENT_COURSE_1NF;
```

Field	Type	Null	Key	Default	Extra
SID	int	NO	PRI	NULL	
SName	varchar(50)	YES		NULL	
Course	varchar(50)	NO	PRI	NULL	
Instructor	varchar(50)	YES		NULL	

4 rows in set (0.01 sec)

```
mysql> CREATE TABLE STUDENT_COURSE_UNF (
->     SID INT,
->     SName VARCHAR(50),
->     Courses VARCHAR(100),
->     Instructor VARCHAR(100)
-> );
```

Query OK, 0 rows affected (0.10 sec)

```
mysql> INSERT INTO STUDENT_COURSE_UNF VALUES
-> (101, 'Ravi', 'DBMS, Python', 'Dr. Rao, Dr. Sharma'),
-> (102, 'Priya', 'AI, DBMS', 'Dr. Kumar, Dr. Rao'),
-> (103, 'Kiran', 'Python', 'Dr. Sharma');
```

Query OK, 3 rows affected (0.02 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM STUDENT_COURSE_UNF;
```

SID	SName	Courses	Instructor
101	Ravi	DBMS, Python	Dr. Rao, Dr. Sharma
102	Priya	AI, DBMS	Dr. Kumar, Dr. Rao
103	Kiran	Python	Dr. Sharma

3 rows in set (0.00 sec)

3. Decompose Relation into 2NF

Aim

To analyze the relation **R(A,B,C,D,E)** with functional dependencies **$A \rightarrow BC$** , **$BC \rightarrow D$** , **$D \rightarrow E$** and determine its 2NF decomposition.

Theory

A relation is in **Second Normal Form (2NF)** if:

1. It is already in 1NF.
2. There is no partial dependency of a non-prime attribute on a proper subset of a candidate key.

A **partial dependency** occurs when a non-key attribute depends on only part of a composite candidate key.

Given:

R(A,B,C,D,E)

Functional dependencies:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

Step 1: Find Candidate Key

Calculate A^+ :

$A^+ = \{A\}$

Using:

$A \rightarrow B, C$

we get:

$A^+ = \{A, B, C\}$

Using:

$BC \rightarrow D$

we get:

$A^+ = \{A, B, C, D\}$

Using:

$$D \rightarrow E$$

we get:

$$A^+ = \{A, B, C, D, E\}$$

Therefore:

$$A^+ = \{A, B, C, D, E\}$$

Since A determines all attributes, A is a candidate key.

Step 2: Check Partial Dependency

The candidate key consists of only one attribute:

Candidate Key = A

A partial dependency can occur only when the candidate key contains more than one attribute.

Since the candidate key contains only A, there is no proper subset of the candidate key.

Therefore, **there is no partial dependency**.

Hence, the relation is already in **2NF**.

Important Observation

The dependency:

$$D \rightarrow E$$

causes a transitive dependency:

$$A \rightarrow \dots \rightarrow D \rightarrow E$$

This is a concern when converting to **3NF**, not **2NF**.

Result

The relation:

$$R(A, B, C, D, E)$$

is already in **2NF** because its candidate key is the single attribute **A** and no partial dependency exists.

```
mysql> CREATE TABLE R (
->   A INT PRIMARY KEY,
->   B VARCHAR(30),
->   C VARCHAR(30),
->   D VARCHAR(30),
->   E VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO R VALUES
-> (1, 'B1', 'C1', 'D1', 'E1'),
-> (2, 'B2', 'C2', 'D2', 'E2'),
-> (3, 'B3', 'C3', 'D3', 'E3');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM R;
```

A	B	C	D	E
1	B1	C1	D1	E1
2	B2	C2	D2	E2
3	B3	C3	D3	E3

```
3 rows in set (0.00 sec)
```

```
mysql> DESCRIBE R;
```

Field	Type	Null	Key	Default	Extra
A	int	NO	PRI	NULL	
B	varchar(30)	YES		NULL	
C	varchar(30)	YES		NULL	
D	varchar(30)	YES		NULL	
E	varchar(30)	YES		NULL	

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM R;
```

A	B	C	D	E
1	B1	C1	D1	E1
2	B2	C2	D2	E2
3	B3	C3	D3	E3

```
3 rows in set (0.00 sec)
```

```
mysql> DESCRIBE R;
```

Field	Type	Null	Key	Default	Extra
A	int	NO	PRI	NULL	
B	varchar(30)	YES		NULL	
C	varchar(30)	YES		NULL	
D	varchar(30)	YES		NULL	
E	varchar(30)	YES		NULL	

```
5 rows in set (0.00 sec)
```

4. Apply 3NF Decomposition

Aim

To decompose **EMPLOYEE**(EmpID, EmpName, DeptID, DeptName, ManagerID) into 3NF by removing transitive dependencies.

Theory

A relation is in **Third Normal Form (3NF)** if:

1. It is already in 2NF.
2. There is no transitive dependency of a non-key attribute on a candidate key.

Consider:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Assume the following functional dependencies:

EmpID $\rightarrow$ EmpName
EmpID $\rightarrow$ DeptID
DeptID $\rightarrow$ DeptName
DeptID $\rightarrow$ ManagerID

Therefore:

EmpID $\rightarrow$ DeptID $\rightarrow$ DeptName
EmpID $\rightarrow$ DeptID $\rightarrow$ ManagerID

Here, DeptName and ManagerID depend indirectly on EmpID through DeptID.

This is called a **transitive dependency**.

Step 1: Identify the Candidate Key

EmpID uniquely identifies an employee.

Therefore:

EmpID $\rightarrow$ EmpName, DeptID, DeptName, ManagerID

Hence:

Candidate Key = EmpID

Step 2: Identify Transitive Dependency

The dependency:

$\text{EmpID} \rightarrow \text{DeptID}$
 $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

means:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore, `DeptName` and `ManagerID` are transitively dependent on `EmpID`.

Step 3: Decompose

Create an **EMPLOYEE** relation:

EMPLOYEE(`EmpID`, `EmpName`, `DeptID`)

Create a **DEPARTMENT** relation:

DEPARTMENT(`DeptID`, `DeptName`, `ManagerID`)

Here:

- `EmpID` is the primary key of **EMPLOYEE**.
- `DeptID` is the primary key of **DEPARTMENT**.
- `DeptID` in **EMPLOYEE** acts as a foreign key.

Final Structure

EMPLOYEE

`EmpID`
`EmpName`
`DeptID`

DEPARTMENT

`DeptID`
`DeptName`
`ManagerID`

Result

The original relation was decomposed into **EMPLOYEE** and **DEPARTMENT** relations, eliminating transitive dependencies and achieving **3NF**.

```
mysql> CREATE TABLE EMPLOYEE (
->   EmpID INT PRIMARY KEY,
->   EmpName VARCHAR(50),
->   DeptID INT,
->   DeptName VARCHAR(50),
->   ManagerID INT
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO EMPLOYEE VALUES
-> (101, 'Ravi', 10, 'CSE', 201),
-> (102, 'Priya', 20, 'ECE', 202),
-> (103, 'Kiran', 10, 'CSE', 201),
-> (104, 'Anitha', 30, 'AIML', 203);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM EMPLOYEE;
```

EmpID	EmpName	DeptID	DeptName	ManagerID
101	Ravi	10	CSE	201
102	Priya	20	ECE	202
103	Kiran	10	CSE	201
104	Anitha	30	AIML	203

4 rows in set (0.00 sec)

```
mysql> SELECT
->   e.EmpID,
->   e.EmpName,
->   e.DeptID,
->   d.DeptName,
->   e.ManagerID
-> FROM EMPLOYEE_3NF e
-> JOIN DEPARTMENT d
-> ON e.DeptID = d.DeptID;
```

EmpID	EmpName	DeptID	DeptName	ManagerID
101	Ravi	10	CSE	201
103	Kiran	10	CSE	201
102	Priya	20	ECE	202
104	Anitha	30	AIML	203

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM DEPARTMENT;
```

DeptID	DeptName
10	CSE
20	ECE
30	AIML

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM EMPLOYEE_3NF;
```

EmpID	EmpName	DeptID	ManagerID
101	Ravi	10	201
102	Priya	20	202
103	Kiran	10	201
104	Anitha	30	203

4 rows in set (0.00 sec)

5. Verify BCNF and Decompose

Aim

To determine whether **R(A,B,C,D)** with FDs **AB → C, C → D, D → A** is in BCNF and decompose it if necessary.

Theory

Boyce-Codd Normal Form (BCNF) is a stronger version of 3NF.

A relation is in BCNF if, for every non-trivial functional dependency:

$$\mathbf{X \rightarrow Y}$$

X must be a **superkey**.

Given:

R (A, B, C, D)

AB → C

C → D

D → A

Step 1: Find Candidate Keys

Calculate **AB⁺**:

$$\mathbf{AB^+ = \{A, B\}}$$

Using:

AB → C

we get:

$$\mathbf{AB^+ = \{A, B, C\}}$$

Using:

C → D

we get:

$$\mathbf{AB^+ = \{A, B, C, D\}}$$

Therefore, **AB** is a candidate key.

Similarly:

$$BC^+ = \{B, C\}$$

$C \rightarrow D$:

$$\{B, C, D\}$$

$D \rightarrow A$:

$$\{A, B, C, D\}$$

Therefore BC is also a candidate key.

Similarly:

$$BD^+ = \{B, D\}$$

$D \rightarrow A$:

$$\{A, B, D\}$$

$AB \rightarrow C$:

$$\{A, B, C, D\}$$

Therefore BD is also a candidate key.

Step 2: Check BCNF

Consider:

$$C \rightarrow D$$

C is not a superkey because:

$$C^+ = \{C, D, A\}$$

B is missing.

Therefore, $C \rightarrow D$ violates BCNF.

Also:

$$D \rightarrow A$$

D is not a superkey because:

$$D^+ = \{D, A\}$$

Therefore, $D \rightarrow A$ also violates BCNF.

Hence, the relation is **not in BCNF**.

Step 3: Decompose

Using the dependency:

$$C \rightarrow D$$

decompose into:

R1 (C, D)

R2 (A, B, C)

In R1:

$$C \rightarrow D$$

C is the key.

In R2:

$$AB \rightarrow C$$

AB is the key.

Result

The original relation is **not in BCNF**. It is decomposed into:

R1 (C, D)

R2 (A, B, C)

to remove the BCNF violations.

```
mysql> SELECT * FROM R1_BCNF;
```

C	D
100	1000
200	2000
300	3000

```
3 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM R2_BCNF;
```

A	B	C
1	10	100
2	20	200
3	30	300

```
3 rows in set (0.01 sec)
```



```
mysql> DESCRIBE R1_BCNF;
```

Field	Type	Null	Key	Default	Extra
C	int	NO	PRI	NULL	
D	int	YES		NULL	

```
2 rows in set (0.00 sec)
```



```
mysql> DESCRIBE R2_BCNF;
```

Field	Type	Null	Key	Default	Extra
A	int	NO	PRI	NULL	
B	int	NO	PRI	NULL	
C	int	YES		NULL	

```
3 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE CHASE_R (
    ->     A INT,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)
```



```
mysql> INSERT INTO CHASE_R VALUES
    -> (1, 'B1', 'C1'),
    -> (2, 'B2', 'C2'),
    -> (3, 'B3', 'C3');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```



```
mysql> SELECT * FROM CHASE_R;
```

A	B	C
1	B1	C1
2	B2	C2
3	B3	C3

```
3 rows in set (0.00 sec)
```

6. Lossless-Join Decomposition Using Chase Algorithm

Aim

To experimentally verify whether a given decomposition is **lossless** using the tabular or Chase algorithm.

Theory

When a relation is decomposed into multiple relations, it is important that the original relation can be reconstructed exactly by joining the decomposed relations.

A decomposition is called **lossless-join** if:

The natural join of the decomposed relations produces exactly the original relation.

A decomposition should not produce:

- Loss of information
- Spurious tuples

The **Chase Algorithm** is a systematic tabular method used to verify lossless-join decomposition.

Example

Consider:

$R(A, B, C)$

with decomposition:

$R_1(A, B)$

$R_2(B, C)$

Chase Procedure

Step 1: Create a tableau with one row for each decomposed relation.

Step 2: Assign distinguished symbols to the attributes.

Step 3: Apply the functional dependencies.

Step 4: Whenever an FD requires two values to be equal, replace the corresponding symbols.

Step 5: Continue until no more changes can be made.

Step 6: If a row becomes a complete distinguished row corresponding to the original relation, the decomposition is lossless.

Interpretation

If the final tableau contains:

A	B	C
a	b	c

with the required distinguished symbols, then the decomposition is lossless.

If no row can be made complete, the decomposition may be lossy.

Importance

The Chase Algorithm is useful because it provides a formal method for checking decomposition properties.

Result

The Chase Algorithm can be used to verify that a decomposition is **lossless-join**, meaning the original relation can be reconstructed without generating incorrect or spurious tuples.

```
mysql> CREATE TABLE CHASE_R (  
->   A INT,  
->   B VARCHAR(20),  
->   C VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO CHASE_R VALUES  
-> (1, 'B1', 'C1'),  
-> (2, 'B2', 'C2'),  
-> (3, 'B3', 'C3');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM CHASE_R;  
+-----+-----+-----+  
| A     | B     | C     |  
+-----+-----+-----+  
| 1     | B1    | C1    |  
| 2     | B2    | C2    |  
| 3     | B3    | C3    |  
+-----+-----+-----+  
3 rows in set (0.00 sec)
```

```
mysql> INSERT INTO CHASE_R2
-> SELECT A, C FROM CHASE_R;
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT
->     R1.A,
->     R1.B,
->     R2.C
-> FROM CHASE_R1 R1
-> JOIN CHASE_R2 R2
-> ON R1.A = R2.A;
```

A	B	C
1	B1	C1
2	B2	C2
3	B3	C3

3 rows in set (0.00 sec)

7. Multivalued Dependencies and 4NF

Aim

To identify the multivalued dependencies in **TEACHER(TID, Subject, Hobby)** and decompose the relation into 4NF.

Theory

A **Multivalued Dependency (MVD)** occurs when one attribute determines a set of values of another attribute independently of other attributes.

It is represented as:

$$X \twoheadrightarrow Y$$

Consider:

TEACHER(TID, Subject, Hobby)

Suppose one teacher can teach several subjects and can have several hobbies. The subjects and hobbies are independent of each other.

For example:

TID	Subject	Hobby
T1	DBMS	Reading
T1	DBMS	Music
T1	AI	Reading
T1	AI	Music

The combinations of subjects and hobbies create unnecessary redundancy.

Multivalued Dependencies

The dependencies are:

$TID \twoheadrightarrow Subject$

$TID \twoheadrightarrow Hobby$

This means:

- A teacher can have multiple subjects.
- A teacher can have multiple hobbies.
- Subjects and hobbies are independent.

4NF

A relation is in **Fourth Normal Form (4NF)** if for every non-trivial multivalued dependency:

$$X \twoheadrightarrow Y$$

X is a superkey.

In this relation, TID is not a superkey for the entire relation because one TID can correspond to many subjects and hobbies.

Therefore, the relation violates 4NF.

Decomposition

Create:

```
TEACHER_SUBJECT(TID, Subject)
```

and:

```
TEACHER_HOBBY(TID, Hobby)
```

The first table stores the subjects taught by each teacher.

The second table stores the hobbies of each teacher.

Result

The multivalued dependencies:

```
TID  $\twoheadrightarrow$  Subject
```

```
TID  $\twoheadrightarrow$  Hobby
```

were identified and the relation was decomposed into:

```
TEACHER_SUBJECT(TID, Subject)
```

```
TEACHER_HOBBY(TID, Hobby)
```

Thus, the redundancy caused by independent multivalued attributes is removed and the relations satisfy **4NF**.

```
mysql> SELECT * FROM SUPPLIER_PART;
+-----+-----+
| Supplier | Part |
+-----+-----+
| S1       | P1   |
| S1       | P2   |
| S2       | P1   |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM SUPPLIER_PROJECT;
+-----+-----+
| Supplier | Project |
+-----+-----+
| S1       | J1      |
| S1       | J2      |
| S2       | J1      |
| S2       | J2      |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM PART_PROJECT;
+-----+-----+
| Part | Project |
+-----+-----+
| P1   | J1      |
| P1   | J2      |
| P2   | J1      |
| P2   | J2      |
+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> INSERT INTO SUPPLIER_PART VALUES
-> ('S1','P1'),
-> ('S1','P2'),
-> ('S2','P1');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO SUPPLIER_PROJECT VALUES
-> ('S1','J1'),
-> ('S1','J2'),
-> ('S2','J1'),
-> ('S2','J2');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO PART_PROJECT VALUES
-> ('P1','J1'),
-> ('P1','J2'),
-> ('P2','J1'),
-> ('P2','J2');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT
-> c.COURSE_NAME,
-> e.STUDENT_ID,
-> e.MARKS
-> FROM course c
-> JOIN enrollment e
-> ON c.COURSE_ID = e.COURSE_ID
-> WHERE e.MARKS >
-> (
-> SELECT AVG(e2.MARKS)
-> FROM enrollment e2
-> WHERE e2.COURSE_ID = e.COURSE_ID
-> );
+-----+-----+-----+
| COURSE_NAME | STUDENT_ID | MARKS |
+-----+-----+-----+
| DBMS       | 101       | 95    |
+-----+-----+-----+
1 row in set (0.01 sec)
```


8. Join Dependencies and 5NF

Aim

To demonstrate how join dependencies cause redundancy and decompose a relation into **5NF (Project-Join Normal Form)**.

Theory

Fifth Normal Form (5NF) is also known as **Project-Join Normal Form (PJ/NF)**.

5NF deals with **Join Dependencies (JD)**.

A join dependency exists when a relation can be reconstructed by joining multiple projections of the relation.

Consider:

`SPJ(Supplier, Part, Project)`

The relation stores information about:

- Which supplier supplies which part.
- Which supplier works on which project.
- Which part is used in which project.

If these relationships are independent, storing all three attributes together may cause redundancy.

Example

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

The same supplier-part and supplier-project information may be repeated in several tuples.

Decomposition

The relation can be decomposed into:

`SUPPLIER_PART(Supplier, Part)`

`SUPPLIER_PROJECT(Supplier, Project)`

`PART_PROJECT(Part, Project)`

The original relation can be reconstructed by joining these relations when the corresponding join dependency holds.

5NF Condition

A relation is in 5NF if every non-trivial join dependency is implied by the candidate keys of the relation.

Importance of 5NF

5NF helps to:

- Remove join-related redundancy.
- Prevent unnecessary duplication.
- Maintain consistency.
- Decompose complex relationships into smaller relations.

Result

The join dependency was identified and the relation was decomposed into:

SUPPLIER_PART
SUPPLIER_PROJECT
PART_PROJECT

Thus, redundancy caused by join dependencies is reduced and the relation can satisfy **5NF**.

```
mysql> INSERT INTO CHASE_R2
-> SELECT A, C FROM CHASE_R;
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT
->     R1.A,
->     R1.B,
->     R2.C
-> FROM CHASE_R1 R1
-> JOIN CHASE_R2 R2
-> ON R1.A = R2.A;
```

A	B	C
1	B1	C1
2	B2	C2
3	B3	C3

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM TEACHER_SUBJECT;
```

TID	Subject
101	DBMS
101	Python
102	AI
102	ML

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM TEACHER_HOBBY;
```

TID	Hobby
101	Cricket
101	Music
102	Reading

```
3 rows in set (0.00 sec)
```

9. Identify Anomalies and Normalize to 3NF

Aim

To identify insertion, deletion, and update anomalies in a poorly designed college database schema and normalize it into 3NF.

Theory

Poor database design can cause **data redundancy** and three major anomalies:

1. Insertion anomaly
2. Update anomaly
3. Deletion anomaly

Consider:

```
COLLEGE_STUDENT (  
  Student_ID,  
  Student_Name,  
  Course_ID,  
  Course_Name,  
  Instructor,  
  Instructor_Phone  
)
```

1. Insertion Anomaly

An insertion anomaly occurs when new information cannot be inserted without providing unnecessary information.

For example, suppose the college introduces a new course but no student has enrolled yet.

If the database requires Student_ID and Student_Name, the course cannot be stored independently.

This is an insertion anomaly.

2. Update Anomaly

Suppose the same instructor teaches several students.

The instructor's phone number may appear in multiple rows.

If the phone number changes, every row containing the instructor's information must be updated.

If one row is not updated, inconsistent information occurs.

This is an update anomaly.

3. Deletion Anomaly

Suppose only one student is enrolled in a particular course.

If that student's record is deleted, information about the course and instructor may also be lost.

This is a deletion anomaly.

Normalization

To eliminate these problems, divide the relation into smaller relations.

STUDENT

```
STUDENT (  
  Student_ID,  
  Student_Name  
)
```

COURSE

```
COURSE (  
  Course_ID,  
  Course_Name,  
  Instructor,  
  Instructor_Phone  
)
```

ENROLLMENT

```
ENROLLMENT (  
  Student_ID,  
  Course_ID  
)
```

Here:

- Student_ID is the primary key of STUDENT.
- Course_ID is the primary key of COURSE.
- Student_ID and Course_ID together form the primary key of ENROLLMENT.
- Student_ID and Course_ID are foreign keys.

Final Structure

```
STUDENT  
Student_ID  
Student_Name
```

```
COURSE  
Course_ID  
Course_Name  
Instructor  
Instructor_Phone
```

ENROLLMENT
Student_ID
Course_ID

Result

The poorly designed college relation was analyzed and insertion, deletion, and update anomalies were identified. The relation was decomposed into **STUDENT**, **COURSE**, and **ENROLLMENT** tables to reduce redundancy and achieve 3NF.

```
mysql> INSERT INTO COLLEGE_STUDENT VALUES
-> (101, 'Ravi', 'C01', 'DBMS', 'Dr. Kumar', '9876543210'),
-> (102, 'Anu', 'C01', 'DBMS', 'Dr. Kumar', '9876543210'),
-> (103, 'Priya', 'C02', 'Python', 'Dr. Meena', '9876543211'),
-> (104, 'Arun', 'C02', 'Python', 'Dr. Meena', '9876543211');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM COLLEGE_STUDENT;
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Course_ID | Course_Name | Instructor | Instructor_Phone |
+-----+-----+-----+-----+-----+-----+
| 101 | Ravi | C01 | DBMS | Dr. Kumar | 9876543210 |
| 102 | Anu | C01 | DBMS | Dr. Kumar | 9876543210 |
| 103 | Priya | C02 | Python | Dr. Meena | 9876543211 |
| 104 | Arun | C02 | Python | Dr. Meena | 9876543211 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> UPDATE COLLEGE_STUDENT
-> SET Instructor_Phone = '9999999999'
-> WHERE Instructor = 'Dr. Kumar';
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2 Changed: 2 Warnings: 0

mysql> SELECT * FROM COLLEGE_STUDENT;
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Course_ID | Course_Name | Instructor | Instructor_Phone |
+-----+-----+-----+-----+-----+-----+
| 101 | Ravi | C01 | DBMS | Dr. Kumar | 9999999999 |
| 102 | Anu | C01 | DBMS | Dr. Kumar | 9999999999 |
| 103 | Priya | C02 | Python | Dr. Meena | 9876543211 |
| 104 | Arun | C02 | Python | Dr. Meena | 9876543211 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT
-> S.Student_ID,
-> S.Student_Name,
-> C.Course_ID,
-> C.Course_Name,
-> C.Instructor,
-> C.Instructor_Phone
-> FROM STUDENT S
-> JOIN ENROLLMENT E
-> ON S.Student_ID = E.Student_ID
-> JOIN COURSE C
-> ON E.Course_ID = C.Course_ID;
+-----+-----+-----+-----+-----+-----+
| Student_ID | Student_Name | Course_ID | Course_Name | Instructor | Instructor_Phone |
+-----+-----+-----+-----+-----+-----+
| 101 | Ravi | C01 | DBMS | Dr. Kumar | 9999999999 |
| 102 | Anu | C01 | DBMS | Dr. Kumar | 9999999999 |
| 103 | Priya | C02 | Python | Dr. Meena | 9876543211 |
| 104 | Arun | C02 | Python | Dr. Meena | 9876543211 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM STUDENT;
```

Student_ID	Student_Name
101	Ravi
102	Anu
103	Priya
104	Arun

```
4 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> SELECT * FROM COURSE;
```

Course_ID	Course_Name	Instructor	Instructor_Phone
C01	DBMS	Dr. Kumar	9999999999
C02	Python	Dr. Meena	9876543211

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM ENROLLMENT;
```

Student_ID	Course_ID
101	C01
102	C01
103	C02
104	C02

```
4 rows in set (0.00 sec)
```

10. Attribute Closure and Candidate Keys

Aim

To compute the closure of attribute sets using functional dependencies and determine all candidate keys of a relation.

Theory

The **attribute closure** of an attribute set X is represented by:

X⁺

It contains all attributes that can be functionally determined from X using the given functional dependencies.

Attribute closure is mainly used to:

- Find candidate keys.
- Verify superkeys.
- Check functional dependencies.
- Analyze normalization.

Given Relation

Consider:

R (A, B, C, D, E)

Functional dependencies:

A → B

B → C

CD → E

E → A

Find A⁺

Start with:

A⁺ = {A}

Using:

A → B

we get:

A⁺ = {A, B}

Using:

$$B \rightarrow C$$

we get:

$$A^+ = \{A, B, C\}$$

We cannot obtain D because no dependency gives D.

Therefore:

$$A^+ = \{A, B, C\}$$

Since A^+ does not contain all attributes, A is not a candidate key.

Find AD⁺

Start:

$$AD^+ = \{A, D\}$$

Using $A \rightarrow B$:

$$AD^+ = \{A, B, D\}$$

Using $B \rightarrow C$:

$$AD^+ = \{A, B, C, D\}$$

Using $CD \rightarrow E$:

$$AD^+ = \{A, B, C, D, E\}$$

Therefore:

$$AD^+ = \{A, B, C, D, E\}$$

Hence AD is a superkey.

Since neither A nor D alone determines all attributes, AD is minimal.

Therefore:

AD is a candidate key.

Find DE⁺

Start:

$$DE^+ = \{D, E\}$$

Using:

$$E \rightarrow A$$

we get:

$$DE^+ = \{D, E, A\}$$

Using:

$$A \rightarrow B$$

we get:

$$DE^+ = \{D, E, A, B\}$$

Using:

$$B \rightarrow C$$

we get:

$$DE^+ = \{D, E, A, B, C\}$$

Therefore:

$$DE^+ = \{A, B, C, D, E\}$$

Hence DE is a superkey.

Since D and E individually do not determine all attributes, DE is minimal.

Therefore:

DE is a candidate key.

Final Candidate Keys

AD

DE

Result

The closures of the required attribute sets were computed using the given functional dependencies. The candidate keys identified for the relation are **AD and DE**.

```
mysql> SELECT A, B, C, D, E
-> FROM R;
```

A	B	C	D	E
1	2	3	4	5
6	7	8	9	10
11	12	13	14	15

```
3 rows in set (0.00 sec)
```



```
mysql> SELECT DISTINCT A, D
-> FROM R;
```

A	D
1	4
6	9
11	14

```
3 rows in set (0.01 sec)
```



```
mysql> SELECT DISTINCT D, E
-> FROM R;
```

D	E
4	5
9	10
14	15

```
3 rows in set (0.00 sec)
```

```
mysql> INSERT INTO v1
-> VALUES (11,160000);
Query OK, 1 row affected (0.01 sec)
```



```
mysql> SELECT * FROM v1;
```

employee_id	salary
1	120000.00
2	150000.00
3	170000.00
4	140000.00
10	180000.00
11	160000.00

```
6 rows in set (0.00 sec)
```



```
mysql> DELETE FROM v1
-> WHERE employee_id = 11;
Query OK, 1 row affected (0.01 sec)
```



```
mysql> SELECT * FROM v1;
```

employee_id	salary
1	120000.00
2	150000.00
3	170000.00
4	140000.00
10	180000.00

```
5 rows in set (0.00 sec)
```